

Alpaca AI Trading Agents Hackathon - Submission Write-Up

Team: Oresti Janko (solo) | **Strategy:** Options Pairs Trading with Statistical Risk Gates

Strategy & AI Logic

Built an autonomous trading agent implementing statistical pairs trading, expressed through defined-risk options spreads rather than raw equity positions. The agent scans a curated universe of ~348 liquid large/mid-cap S&P 500 stocks daily, verified live against Alpaca's own tradable-assets API, and identifies pairs whose prices are historically correlated (≥ 0.80) and cointegrated (Engle-Granger test, $p \leq 0.10$) over a 60-day lookback window.

When a pair's price spread diverges beyond a statistical threshold (z-score ≥ 1.5 , deliberately loosened from a conventional 2.0+ to generate enough trade opportunities within the competition's 7-day window), the agent expresses the resulting directional view through options: a bull call spread on the side expected to outperform, and a bear put spread on the side expected to underperform. Both are defined-risk, multi-leg structures, the maximum possible loss on any trade is capped by construction, not just by a stop order that might not fire in time.

Risk Gates

- **Position sizing:** 2.5% of capital per trade
- **Ticker-overlap protection:** no underlying stock can be committed to more than one open pair at once, preventing unintended concentration
- **Stop-loss:** exit if the spread's z-score moves to 3.0 against the position, or if it hasn't reverted within 3 days (whichever comes first)
- **Take-profit:** locks in gains at 35% of premium risked, checked every 30 minutes independent of the daily cycle
- **Portfolio drawdown circuit-breaker:** soft warning at 8% (halves new position sizing), hard halt at 12% (blocks all new trades)
- **Broker reconciliation:** the system's own record of open positions is automatically checked against what Alpaca actually holds, so a partial fill or failed close can't leave a position invisibly unmanaged

Alpaca Infrastructure

Built as a standalone Python agent (not a chat-driven loop) using Alpaca's Trading API and Market Data API directly, chosen for auditability and testability over each piece of decision logic. Runs autonomously via Windows Task Scheduler: a daily cycle evaluates entries/exits,

and a 30-minute cycle independently monitors drawdown and take-profit targets. All option spreads are placed as genuine multi-leg (MLEG) orders.

Results

- Starting capital: \$100,000
- Current equity: **\$94,449.87**
- Total return: **-5.55%**
- Position market value at close: \$14,681 long, -\$6,870 short (net options exposure)
- 6 pairs open as of final day; several additional pairs opened and closed earlier in the week via take-profit, time-stop, or the fill-verification safety net (see Engineering Process below)
- Peak observed drawdown: 5.55%, against a designed hard halt of 12% - the circuit-breaker was never triggered
- Alpaca paper account ID: **PA3BY9COKHGN**

Honest framing: this strategy finished the week with a net loss, not a gain. Rather than adjust the numbers or the narrative after the fact, the results are reported as they stand. What the week demonstrates is not "the strategy is guaranteed to be profitable", a single 7-day sample is far too short and noisy to claim that for any real strategy, but that a statistically-grounded approach with genuinely enforced risk controls can operate autonomously, survive real production issues, and stay well inside its own designed risk limits throughout, which was the actual engineering goal.

Engineering Process & Honesty About Limitations

Live trading surfaced real issues that no amount of dry-run testing caught, each was diagnosed and fixed during the competition window rather than hidden:

- **Fill verification gap:** early positions were recorded as "open" immediately after order submission, without confirming every leg actually filled at the broker, fixed by polling for real fills before writing to state, and unwinding any partial fill automatically.
- **Wrong close target:** closing positions by underlying ticker (rather than the actual option contract symbol) meant close attempts silently failed for options-based positions, fixed with a dedicated multi-leg close path addressed by real contract symbols.
- **Strike collision on wide strike-increment stocks:** both legs of a spread could independently round to the same real listed strike on certain underlyings, fixed by tracking claimed strikes within a spread and falling back to the next-closest distinct one.
- **Ticker-overlap / concentration risk:** discovered that one underlying (CBRE) had been committed to three separate concurrent pairs simultaneously, effectively tripling intended exposure to that name, fixed with an explicit overlap check before any new entry is opened.

Known limitations, stated honestly: no formal backtest was run (given limited predictive value for the specific trading week and time constraints); hedge ratios are static (computed

once via OLS, not dynamically updated); bid-ask slippage isn't precisely modeled; and one position (MCK) showed strike/price data that appeared inconsistent with the real underlying price, suggesting a possible data-staleness issue not fully root-caused before the deadline, flagged rather than concealed.

Full technical documentation, bug log, and build history available in the repository ([DOCUMENTATION.md](#)).